

Q.No:7

Question description

There are n cities and initially no roads between them. However, every day a new road will be constructed, and there will be a total of m roads.

A component is a group of cities where there is a route between any two cities using the roads. After each day, your task is to find the number of components and the size of the largest component.

Input

The first input line has two integers n and m : the number of cities and roads. The cities are numbered $1, 2, \dots, n$.

Then, there are m lines describing the new roads. Each line has two integers a and b : a new road is constructed between cities a and b .

You may assume that every road will be constructed between two different cities.

Output

Print m lines: the required information after each day.

Constraints

- $1 \leq n \leq 10^5$
- $1 \leq m \leq 2 \cdot 10^5$
- $1 \leq a, b \leq n$

✓ Logical Test Cases

Test Case 1

INPUT (STDIN)

```
5 6
1 2
2 3
3 1
3 4
4 5
5 4
```

EXPECTED OUTPUT

```
4 2
3 3
3 3
2 4
1 5
1 5
```

Test Case 2

INPUT (STDIN)

```
5 5
1 3
2 4
3 1
3 2
5 4
```

EXPECTED OUTPUT

```
4 2
3 2
3 2
2 4
1 5
```

Code:

```
#include <stdio.h>

#define MAX 100005

int parent[MAX];
int sz[MAX];

int components;
int largest;
int find(int x)
```

```
{
if (parent[x] != x)
{
parent[x] = find(parent[x]);
}

return parent[x];
}

int join(int i, int j)
{
int root1 = find(i);
int root2 = find(j);
if (root1 == root2)
{
return 0;
}

/* Union by size */
if (sz[root1] < sz[root2])
{
int temp = root1;
root1 = root2;
root2 = temp;
}

parent[root2] = root1;
sz[root1] = sz[root1] + sz[root2];

components--;

if (sz[root1] > largest)
{
largest = sz[root1];
}

return 1;
}

int main()
{
int n, m;
int i;
int a, b;

scanf("%d %d", &n, &m);
for (i = 1; i <= n; i++)
{
parent[i] = i;
sz[i] = 1;
}

components = n;
largest = 1;
```

```
printf("\nOutput:\n");
for (i = 0; i < m; i++)
{
scanf("%d %d", &a, &b);

join(a, b);

printf("%d %d\n", components, largest);
}

return 0;
}
```

Output:

Test_case1:

```
aadityajaiswal@Aad:
5 6
1 2
2 3
3 1
3 4
4 5
5 4
Output:
4 2
3 3
3 3
2 4
1 5

1 5
aadityajaiswal@Aad:
```

Test_case2:

```
aadityajaiswal@Aad:
5 5
1 3
2 4
3 1
3 2
5 4
Output:
4 2
3 2
3 2
2 4

1 5
aadityajaiswal@Aad:
```